

AIOps Assessment – Basic AIOps Monitoring & Event Processing

Objective

In this assessment, you will work with a **Python-based AIOps simulation** to analyse operational data, identify abnormal behaviour, process anomaly events, and demonstrate a basic AIOps workflow.

You will work directly in the provided GitHub repository using GitHub Codespaces / VS Code.

Your completed GitHub repository, documentation, execution results, and submitted evidence will be used for assessment.

Scenario

You are part of a DevOps team responsible for monitoring an application service.

The service produces operational information in the form of **metrics and logs**. The operations team wants to identify unusual behaviour and process detected anomalies as events.

Your task is to work with the provided AIOps simulation and demonstrate the following workflow:

Operational Data → Anomaly Detection → Event Generation → Producer → Topic → Consumer → AIOps Output

The assessment uses a lightweight Python-based simulation rather than a real Kafka or Airflow deployment.

Part 1: Set Up and Understand the Environment

Task 1: Set Up the Project

1. Open the provided [GitHub repository](#) and create a **GitHub Codespace** using the default configuration.
2. Confirm that you are working in **your copy of the exercise repository**, not the original repository.
3. Wait for Visual Studio Code to load.
4. Explore the repository structure and identify the files/components related to:
 - operational data
 - metrics and logs
 - anomaly detection
 - event production
 - event topics
 - event consumption
 - final AIOps processing
5. Review the provided source files and documentation to understand the purpose of each component.
6. In the README, provide a brief description of:
 - the service being monitored
 - the operational problem being addressed

- the purpose of AIOps in this assessment

Do not rewrite or replace the provided application components.

Check your work

Your fork should contain the original assessment structure, your README should describe the AIOps scenario, and you should be able to explain the purpose of the major components before executing the workflow.

Part 2: Prepare and Inspect Operational Data

Task 2: Analyse Logs and Metrics

The repository contains a small set of synthetic operational data representing the behaviour of an application service.

Inspect the available data and determine:

1. Which fields represent metrics.
2. Which fields represent log information.
3. How timestamps are used in the operational data.
4. Which observations appear to represent normal behaviour.
5. Which observations appear to represent unusual behaviour.

Document your observations in the README or designated assessment response file.

Your analysis should be based on the data provided in the repository.

Part 3: Validate Anomaly Detection & Event Streaming Workflow

Task 3: Identify Anomalies

Use the provided anomaly-detection component to analyse the operational data.

Your task is to configure or use the provided detection mechanism according to the requirements of the assessment.

Verify that the detection process:

1. Processes the available operational data.
2. Identifies abnormal metric behaviour.
3. Identifies relevant concerning log events.
4. Distinguishes normal observations from anomalous observations.
5. Produces a readable result identifying the detected anomalies.
6. Provides sufficient information to understand why an observation was flagged.

Do not replace the provided detection architecture with your own implementation.

Analysis

Review the resulting detection report.

Record:

- the anomalies detected
- the relevant metric/log information
- whether any expected anomaly was missed
- whether any normal event was incorrectly flagged

Briefly describe one limitation or possible improvement to the detection approach.

Task 4: Verify the AIOps Event Flow

The repository contains a lightweight **Python-based simulation of an event-streaming system**. Use the provided components to validate the flow of anomaly events.

Verify that:

1. An anomaly identified by the detection process results in an event.
2. The event is passed to the producer.
3. The producer publishes the event to the appropriate topic.
4. The consumer receives the event from the topic.
5. The consumer processes the received event.
6. The processed event reaches the downstream AIOps component.

You must be able to identify the role of the following components in the workflow:

- Producer, Topic, Consumer, Event/message

Execution

Run the provided workflow and verify that an anomaly can travel through the complete event-processing pipeline.

Record the result of your execution.

Part 4: Troubleshoot & Demonstrate the AIOps Workflow

Task 5: Investigate and Correct the Workflow

The provided assessment environment contains issues that prevent the complete AIOps workflow from operating as expected.

Investigate the workflow and identify the problems.

For each problem you identify:

1. Determine which component is affected.
2. Identify the cause.
3. Apply the appropriate correction.
4. Execute the affected component again.
5. Verify that the correction resolves the issue.

Your corrections should work within the existing architecture.

Do not replace the provided components with an unrelated implementation.

Task 6: Execute the End-to-End Pipeline

After completing your investigation and corrections, execute the complete AIOps workflow.

Demonstrate the complete flow:

Operational Data → Anomaly Detection → Event → Producer → Topic → Consumer → AIOps Output

Verify that:

1. Operational data is processed.
2. Anomalous behaviour is detected.
3. An anomaly event is generated.
4. The event is published.

5. The event is consumed.
6. The event is processed successfully.
7. The final output represents the detected operational issue.

Part 5: Document Your Findings

Task 7: Update the README

Update the project README with the information required to reproduce your work.

Your README must include:

1. A brief explanation of the AIOps scenario.
2. A description of the operational data.
3. Your observations from the logs and metrics.
4. Your anomaly-detection findings.
5. A description of the event-processing flow.
6. The result of your final workflow execution.
7. Any issues you identified and corrected.
8. At least one limitation or possible improvement to the implemented AIOps approach.
9. The steps required for another user to reproduce your demonstration.

Do not include screenshots in place of the required written explanations.

Part 6: Validate & Save Your Work

Task 8: Run the Provided Validation

Use the tests or validation mechanisms provided in the repository to verify your implementation.

Confirm that:

- the operational data can be processed
- anomaly detection behaves as expected
- anomaly events are generated
- events can move through the simulated event pipeline
- consumers can process the generated events
- the final AIOps workflow completes successfully

Investigate and resolve any failures before submission.

Task 9: Commit and Push Your Changes

Before submitting:

1. Review all files you have modified.
2. Ensure that only relevant assessment changes are included.
3. Commit your changes using a meaningful commit message.
4. Push your changes to your GitHub fork.
5. Confirm that the latest version of your work is visible on GitHub.

Submission Requirements

Submit the following:

1. GitHub Repository

Provide the URL of your completed GitHub repository.

2. Pull Request

Create a pull request from your fork to the original exercise repository.

In the pull request description, provide:

- what the AIOps workflow detects
- how you validated the workflow
- the result of the final execution
- issues you identified and corrected
- one known limitation or possible improvement

3. Evidence

Provide screenshots showing:

- operational data/logs/metrics being analysed
- anomaly-detection results
- successful event generation
- producer → topic → consumer flow
- final AIOps output
- successful validation/test execution

4. Repository Contents

Your final repository should contain the required:

- source files
- operational data
- tests/validation files
- documentation
- configuration changes

Final Verification

Before submitting, verify that:

- Your repository is your GitHub fork.
- Your latest changes have been committed.
- Your changes have been pushed to GitHub.
- The README contains the required documentation.
- The operational data can be processed successfully.
- Anomalies can be identified.
- Anomaly events are generated.
- Events successfully move through the simulated producer/topic/consumer flow.
- The final AIOps workflow executes successfully.
- Your screenshots represent your final implementation.
- Your pull request contains the required information.
- Another person can understand and reproduce your workflow using your README.

Important Instructions

- Complete the assessment independently.
- Use the provided AIOps architecture and components.
- Do not replace the provided architecture with a different implementation.
- Do not artificially generate anomaly results.
- Do not modify data solely to produce a desired result.
- Do not bypass validation or testing mechanisms.
- Do not introduce external infrastructure such as Kafka, Airflow, or cloud services.
- Keep your implementation within the provided Python-based simulation.
- All required changes must be committed and pushed to GitHub.
- Your **GitHub repository, pull request, documentation, and submitted evidence** will be used for assessment.